

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО» (Университет
ИТМО)

Факультет: Факультета прикладной информатики

Образовательная программа:

Программирование в инфокоммуникационных системах

Направление подготовки (специальность): 09.03.03

Прикладная информатика

О Т Ч Ё Т

по выпускной квалификационной работе

Тема: Разработка архитектуры нейросетевого агента на основе слоев

Mamba-2 для решения задач в среде VizDoom

Выполнил: Федорин К.В.

Научный
руководитель: Гусарова
Наталья Фёдоровна

Санкт-Петербург

2025

СОДЕРЖАНИЕ

ЗАДАНИЕ.....	3
АННОТАЦИЯ.....	5
ВВЕДЕНИЕ.....	6
Глава 1 Исследование предметной области.....	9
1.1 Описание предметной области.....	9
1.1.1 Обучение с подкреплением (Reinforcement Learning).....	9
1.1.2 Визуальная навигация в частично наблюдаемых средах.....	9
1.1.3 Архитектуры памяти для обработки последовательностей.....	10
1.2 Исследование альтернативных алгоритмов и архитектур.....	12
1.2.1 RL алгоритмы для обучения.....	12
1.2.1.1 PPO.....	12
1.2.1.2 SAC.....	12
1.2.1.3 DM.....	13
1.2.1.4 Итог.....	13
1.2.2 Архитектуры нейронных сетей.....	14
1.2.2.1 CNN + MLP.....	14
1.2.2.2 CNN + Transformer/Mamba/GRU.....	14
1.2.2.3 VLA.....	14
1.2.2.4 Предлагаемый гибридный подход.....	15
1.2.2.5 Вывод.....	15
1.3 Сравнительная характеристика моделей лабораторий.....	16
Глава 2 Формулирование требований к системе.....	17
2.1 Функциональные требования.....	18
2.2 Нефункциональные требования.....	19
Глава 3 Проектирование системы.....	20
3.1 Разработка архитектуры системы обучения.....	21
3.1.1. Диаграмма компонентов системы (UML).....	21
3.1.2 Диаграмма последовательности цикла "Агент-Среда" (UML).....	21
3.1.3 Модель процесса проведения исследования (BPMN).....	23
2.4. Схема архитектуры RL-агента.....	24
3. Модели состояния AS-IS и TO-BE.....	25
3.1. Текущее состояние (AS-IS).....	25
3.2. Целевое состояние (TO-BE).....	25
Глава 4 Разработка.....	25
4.1 Настройка окружения и изучение архитектуры Sample Factory.....	26
4.2 Интеграция новых архитектур в Sample Factory.....	27

4.2.1 Интерфейс модуля памяти.....	27
4.2.2 Реализация TransformerCore.....	27
4.2.3 Реализация Mamba2Core.....	29
4.2.4 Реализация PerceiverCore.....	30
4.3 Экспериментальная процедура.....	32
4.3.1 Конфигурация экспериментов.....	32
4.3.2 Метрики оценки.....	32
4.4 Результаты экспериментов.....	34
4.4.1 Пилотное обучение (10 млн шагов).....	34
4.4.2 Полное обучение (500 млн шагов).....	34
4.4.3 Сравнение Mamba-1, Mamba-2 и GRU.....	36
4.5 Обсуждение результатов.....	39
ЗАКЛЮЧЕНИЕ — 1 стр.....	40
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	41
ПРИЛОЖЕНИЯ.....	43

ЗАДАНИЕ

Техническое задание: исследовать и сравнить четыре архитектуры памяти (GRU, Transformer, Mamba-2, Perceiver IO) для агентов обучения с подкреплением в среде ViZDoom.

Исходные данные: окружение ViZDoom, фреймворк Sample Factory, базовая архитектура GRU с 512-мерным скрытым состоянием, вычислительный ресурс NVIDIA RTX 5090 (24 ГБ VRAM).

Цель работы: выполнить эмпирическое исследование, сравнивающее производительность четырёх архитектур по метрикам скорость обучения, финальная производительность агента, латентность инференса и стабильность обучения.

Основные задачи:

- Настройка окружения, установка зависимостей, изучение архитектуры Sample Factory
- Реализация трёх архитектур (TransformerCore, Mamba2Core, PerceiverCore) с интеграцией в Sample Factory и юнит-тестами
- Пилотные эксперименты на 10М шагов обучения для быстрой оценки архитектур
- Полное обучение на 500М шагов для получения финальных результатов
- Анализ результатов и написание ВКР

Подлежащие разработке вопросы: как интегрировать новые архитектуры в Sample Factory с совместимым интерфейсом, какой размер буфера состояния оптимален для каждой архитектуры, сколько итераций нужно для статистической значимости, как выбрать между полным

обучением 500M шагов или сокращённым 250M, как максимизировать использование GPU и обработать отказы оборудования.

АННОТАЦИЯ

В настоящей выпускной квалификационной работе исследуется проблема выбора архитектуры памяти для интеллектуальных агентов, обученных методом с подкреплением для решения задач визуальной навигации в частично наблюдаемой среде ViZDoom. Проведено эмпирическое сравнение четырёх архитектур обработки последовательностей -- GRU, Transformer, Mamba-2 и Perceiver IO -- интегрированных в фреймворк Sample Factory на основе алгоритма PPO.

В ходе работы реализованы три новых модуля памяти (TransformerCore, Mamba2Core, PerceiverCore) с обеспечением совместимого интерфейса с базовой архитектурой Sample Factory. Проведены пилотные эксперименты (10 млн шагов) и полное обучение (до 50 млн шагов) на картах ViZDoom с использованием GPU NVIDIA RTX 5090.

Результаты показывают, что архитектуры на основе Mamba-2 демонстрируют наилучший баланс между скоростью обучения, финальной производительностью агента и вычислительной эффективностью. Трансформерная архитектура и Perceiver IO уступают по стабильности обучения и требуют значительно больше ресурсов. Полученные метрики включают скорость сходимости, финальный reward, латентность инференса и потребление VRAM.

Работа содержит практические рекомендации по выбору архитектуры памяти для RL-агентов в зависимости от ограничений на вычислительные ресурсы.

ВВЕДЕНИЕ

Задача визуальной навигации в условиях частичной наблюдаемости является одной из ключевых проблем при разработке интеллектуальных агентов, функционирующих в сложных динамических средах. Визуальная навигация предполагает принятие решений исключительно на основе визуального входа без доступа к точным координатам, глобальной карте или полной информации о состоянии среды. Подобные условия характерны как для автономных робототехнических систем, так и для виртуальных игровых сред.

Игровые трёхмерные среды, в частности игра DOOM и её исследовательская платформа ViZDoom, широко применяются в научных исследованиях в области обучения с подкреплением. Они предоставляют контролируемую, воспроизводимую и достаточно сложную среду, позволяющую исследовать поведение агентов в условиях ограниченного обзора, динамических объектов и необходимости быстрого реагирования.

В данной работе рассматривается задача визуальной навигации и управления агентом в среде DOOM с использованием современных методов машинного обучения, включая обучение с подкреплением и мультимодальные Vision-Language-Action модели.

Актуальность темы исследования обусловлена стремительной трансформацией подходов к созданию интеллектуальных агентов в трехмерных виртуальных средах. В 2025-ом году такие технологические решения, как NVIDIA NitroGen и Google SIMA 2.0, продемонстрировали возможность создания универсальных ИИ-агентов, способных ориентироваться в сложных 3D-пространствах на основе визуального восприятия и естественного языка, подобно человеку.

Несмотря на прогресс нейросетевых моделей, задача эффективной и экономичной навигации в динамических средах остается открытой. Среда DOOM (ViZDoom) продолжает играть роль эталонного полигона для апробации алгоритмов из-за своей высокой динамики и сложности

визуального ряда при относительно низких требованиях к ресурсам. Исследование алгоритмов навигации в этом контексте позволяет не только оптимизировать классические методы поиска пути, но и интегрировать их с современными архитектурами глубокого обучения и генеративного ИИ. Это открывает новые перспективы для создания более адаптивных и реалистичных игровых персонажей, способных действовать в условиях неопределенности, что является критически важным для развития современной индустрии разработки игр (Gamedev) и систем автономного управления».

Цель работы: реализация, экспериментальное сравнение и анализ перспективных алгоритмов визуальной навигации и управления агентом в игровой среде DOOM, а также разработка гибридного подхода, сочетающего преимущества различных методологий.

В рамках данной работы будут сопоставлены и сравнены несколько подходов в обучении ИИ-агента, который будет проходить различные уровни игры DOOM. На основе них в процессе исследования будут предложены новые подходы.

Задачи исследования:

1. Разработать программный комплекс на базе платформы ViZDoom, позволяющий проводить контролируемые эксперименты по навигации агентов в сценариях с различной топологической сложностью и необходимостью долгосрочного планирования (Long-term dependencies).
2. Реализовать и интегрировать в архитектуру RL-агента (например, PPO или SAC) различные модули памяти для сравнительного тестирования:
 - Baseline-модуль: Классическая рекуррентная архитектура (LSTM/GRU).
 - SOTA-модуль: Трансформерная архитектура с механизмом внимания (Attention).

- Исследовательский модуль: Селективная модель пространства состояний (Mamba-2).
3. Провести серию экспериментов (бенчмаркинг) по обучению агентов, фиксируя ключевые метрики эффективности: скорость сходимости обучения, среднюю длину пройденного пути, точность достижения цели в лабиринтах и затраты вычислительных ресурсов (FPS, использование VRAM).
 4. Выполнить кросс-архитектурный анализ полученных результатов, оценив степень превосходства (или границы применимости) модели Mamba-2 над классическими и трансформерными решениями в специфических условиях движка DOOM.
 5. Сформулировать практические рекомендации
 6. Подготовить документацию по проекту и сформулировать выводы о проделанной работе.

Глава 1 Исследование предметной области

1.1 Описание предметной области

Предметной областью настоящего исследования является пересечение трех динамично развивающихся направлений в сфере искусственного интеллекта: обучение с подкреплением (Reinforcement Learning, RL), компьютерное зрение (Computer Vision) и архитектуры нейронных сетей для обработки последовательностей. Центральным объектом исследования выступает интеллектуальный агент, способный к автономной навигации в сложной, частично наблюдаемой трехмерной среде.

1.1.1 Обучение с подкреплением (Reinforcement Learning)

Обучение с подкреплением — это парадигма машинного обучения, в рамках которой агент учится принимать оптимальные решения путем взаимодействия со средой. В отличие от обучения с учителем, где используются размеченные данные, в RL агент получает обратную связь в виде скалярного сигнала — награды (reward). Цель агента — максимизировать кумулятивную награду за эпизод. Ключевыми элементами этой парадигмы являются:

Агент (Agent): Сущность, принимающая решения.

Среда (Environment): Мир, с которым взаимодействует агент.

Состояние (State): Полное или частичное описание среды в конкретный момент времени.

Действие (Action): Решение, принимаемое агентом.

Политика (Policy): Стратегия, которой следует агент для выбора действий в определенных состояниях.

В контексте данной работы используются современные алгоритмы RL, такие как PPO (Proximal Policy Optimization) или SAC (Soft Actor-Critic), которые хорошо зарекомендовали себя в решении сложных задач с непрерывным или большим пространством состояний и действий.

1.1.2 Визуальная навигация в частично наблюдаемых средах

Классические задачи RL часто предполагают, что агент имеет доступ к

полному состоянию среды (полная наблюдаемость). Однако в реальном мире и в сложных симуляциях, таких как видеоигры, агент получает лишь частичную информацию — например, изображение на экране. Такие среды называются частично наблюдаемыми марковскими процессами принятия решений (Partially Observable Markov Decision Processes, POMDP).

Визуальная навигация — это подзадача, в которой агент должен ориентироваться в пространстве, опираясь исключительно на визуальный вход (поток пикселей). Это требует от агента не только распознавания объектов на текущем кадре, но и формирования внутреннего представления о мире, которое учитывает предыдущие наблюдения. Для успешной навигации в лабиринтах или поиска удаленных целей агент должен обладать памятью, позволяющей ему понимать, где он уже был, и строить долгосрочные планы.

В качестве среды для экспериментов используется платформа ViZDoom, представляющая собой программный интерфейс (API) к классической игре DOOM (1993). ViZDoom является стандартом де-факто для исследований в области визуального RL благодаря следующим особенностям:

Сложная 3D-графика: Псевдотрехмерное окружение с динамическим освещением, текстурами и врагами.

Частичная наблюдаемость: Агент видит мир от первого лица, что естественно создает POMDP-условия.

Высокая скорость симуляции: Позволяет проводить тысячи эпизодов обучения за короткое время.

Гибкость: Предоставляет широкие возможности для создания кастомных сценариев (карт) с различными задачами (например, "найди выход из лабиринта", "собери предметы").

1.1.3 Архитектуры памяти для обработки последовательностей

Поскольку агент в POMDP-среде должен принимать решения на основе последовательности наблюдений, ключевую роль в его архитектуре играет модуль памяти. Этот модуль обрабатывает последовательность эмбедингов

(сжатых представлений) кадров, полученных от сверточной нейронной сети (CNN), и формирует скрытое состояние, или "контекст", который используется для принятия решения.

В данном исследовании рассматриваются три класса архитектур памяти:

Рекуррентные нейронные сети (RNN): Классический подход, представленный сетями LSTM (Long Short-Term Memory) и GRU (Gated Recurrent Unit). Они обрабатывают последовательность пошагово, сохраняя внутреннее состояние. Их главный недостаток — проблема затухания или взрыва градиента, что затрудняет запоминание очень длинных зависимостей.

Трансформеры (Transformers): Архитектура, основанная на механизме внимания (attention), которая стала доминирующей в обработке естественного языка. Внимание позволяет модели напрямую сопоставлять элементы в длинных последовательностях, эффективно решая проблему долгосрочных зависимостей. Однако квадратичная вычислительная сложность по отношению к длине последовательности делает их ресурсоемкими.

Модели на основе пространств состояний (State Space Models, SSM): Новый класс моделей, к которому относится Mamba. SSM-архитектуры вдохновлены классической теорией управления и сочетают в себе преимущества RNN (линейная сложность вычислений) и Трансформеров (способность моделировать длинные зависимости). Селективный механизм в Mamba позволяет модели динамически фокусироваться на релевантной информации из прошлого, что делает ее потенциально более эффективной и легковесной по сравнению с Трансформерами.

Таким образом, предметная область исследования находится на стыке обучения с подкреплением, компьютерного зрения и современных нейросетевых архитектур. Работа направлена на решение фундаментальной проблемы навигации в частично наблюдаемых средах путем сравнительного анализа и внедрения передовых моделей памяти, способных обеспечить

баланс между производительностью и вычислительной эффективностью.

1.2 Исследование альтернативных алгоритмов и архитектур

Было проведено исследование работ в области обучения RL-агентов в игровых средах. На основе этого, были проанализированы алгоритмы и архитектуры нейронных сетей, наиболее подходящих для работы с настраиваемой игровой средой ViZDoom.

1.2.1 RL алгоритмы для обучения

1.2.1.1 PPO

Proximal Policy Optimization относится к классу policy-gradient алгоритмов обучения с подкреплением. В рамках PPO оптимизация политики агента производится напрямую, в соответствии с происходящим на сцене, что позволяет избежать неустойчивости обучения и переоценки действий. В контексте среды DOOM алгоритм PPO используется как базовый метод управления агентом на основе визуального входа. Визуальные данные обрабатываются сверточной нейронной сетью, после чего формируется распределение вероятностей над пространством действий с помощью полносвязных слоёв, с возможными модификациями. Потенциально это требует много данных для обучения, что не является ограничением с учётом возможности бесконечной случайной генерации игровых ситуаций.

1.2.1.2 SAC

Soft Actor-Critic – Главная особенность SAC заключается в том, что агент стремится не только получить как можно больше награды, но и действовать максимально непредсказуемо (сохранять высокую энтропию).

В отличие от классических алгоритмов, которые ищут только одну «лучшую» траекторию, SAC поощряет агента исследовать разные варианты действий. Это предотвращает преждевременную сходимость к плохому локальному решению и делает поведение агента более гибким.

Однако этот алгоритм требует большего количества памяти для сохранения накопленного в ходе обучения опыта и корректировки изменения

весов.

1.2.1.3 DM

Decision Mamba – активно развивающийся подход, где RL рассматривается как задача моделирования последовательностей действий агента. Этот подход наиболее эффективен, если есть набор готовых демонстраций (Offline RL). Гибрид DM-H (Mamba + Transformer) показывает превосходство в задачах с долгосрочной памятью, работая в 28 раз быстрее классических решений на длинных горизонтах планирования.

Однако в рамках рассматриваемой задачи, генерация эталонного поведения в игровых сессиях представляется слишком время затратной, что делает этот и подобные ему алгоритмы не релевантными.

1.2.1.4 Итог

В связи с описанными минусами и плюсами модели было решено остановиться на алгоритме PPO, поскольку он является наиболее приспособленным к использованию с виртуальной площадкой ViZDoom. Окончательная сравнительная таблица алгоритмов приведена ниже:

Таблица 1 – Сравнение RL алгоритмов.

Алгоритмы	PPO	SAC	DM
Критерий			
Тип обучения	Из текущего опыта	Из буфера памяти	Моделирование опыта
Стабильность	Высокая	Средняя.	Высокая
Память (VRAM)	Умеренная	Высокая	Низкая

1.2.2 Архитектуры нейронных сетей

1.2.2.1 CNN + MLP

Свёрточная сеть и полносвязный слой является самой базовой архитектурой для этой задачи. Ключевая слабость моделей с ней – плохое запоминание как происходящих событий, так и ключевых игровых паттернов.

1.2.2.2 CNN + Transformer/Mamba/GRU

Такая архитектура закрывает проблему предшественницы. Модель успешно адаптируется к последовательностям игровых событий, стабильнее запоминает желаемое игровое поведение.

1.2.2.3 VLA

Vision-Language-Action модели[1][3][4] представляют собой класс мультимодальных алгоритмов, в которых действия агента формируются на основе объединённого представления визуальных данных и языкового описания задачи. Как правило, такие модели строятся на архитектурах трансформеров/Mamba и обучаются в авторегрессионном режиме или с использованием гибридных схем, включающих элементы обучения с подкреплением.

Преимуществом VLA-подходов является возможность оперировать высокоуровневыми абстракциями и учитывать семантику задачи. Это потенциально позволяет агенту выполнять сложные сценарии поведения и обобщать полученные знания. Однако применение VLA-моделей в среде DOOM осложняется высокой вычислительной сложностью, отсутствием специализированных обучающих наборов данных и слабой интеграцией с существующими бенчмарками ViZDoom. Единственный целесообразный путь её использования – взять предобученную CombatVLA[3] модель, дообучить её на размеченных записях обученных RL-агентов, а затем использовать PPO для оттачивания мастерства модели.

Так как подобные модели нуждаются в большем количестве параметров

для масштабирования и наращивания ключевой разницы с другими архитектурами, то они требуют и большего количества времени, как для обучения, так и для инференса. Это и может стать ключевой слабостью в real-time симуляциях.

1.2.2.4 Предлагаемый гибридный подход

Данная архитектура подразумевает скрещивание двух предыдущих методов. VLA становится долгосрочным планировщиком действий, решающим ключевые задачи ориентации в пространстве и стратегии. Более лёгкая модель решает, что ей делать здесь и сейчас, держа в своём вводе рекомендованную последовательность действий планировщика. Таким образом, более компактная VLA или CNN + Mamba-2 становится моторикой конечного агента.

Очевидно, данная архитектура подразумевает постепенное освоение всех предыдущих этапов и решение всех перечисленных ранее проблем, что делает её чрезвычайно сложной в реализации.

1.2.2.5 Вывод

В соответствии с имеющимися ресурсами и ограничениями продвинутых подходов было решено остановиться на обучении при помощи метода PPO архитектуры из продвинутой сегментационной модели (CNN) совмещённой с Mamba-2 блоками для создания и обучения агента сложным поведенческим паттернам.

1.3 Сравнительная характеристика моделей лабораторий

На данный момент активно ведутся исследования в области выполнения команд агентом в виртуальной среде. Ещё ни одно из них не смогло достичь значимого и полноценного прохождения 3d-видеоигры от начала до самого конца. Все научные коллективы сфокусированы на выполнении моделями ключевых и базовых микросценариев игр: убийство монстра, путь к видимой достопримечательности, защита объекта. Явными примерами этому являются свежие работы флагманских лабораторий искусственного интеллекта:

- 1) Nvidia выпустила Vision-Action модель Nitrogen [1], обученную на видеопрохождениях различных игр, которая пока лишь способна обрабатывать короткие сцены с явно видимой целью.
- 2) Google разработала полноценную VLA модель SiMA2 [4] на основе большой языковой модели Gemini. SiMA2 может следовать заданным текстом инструкциям, но, в силу своего размера и неповоротливости, она плохо проявляется в динамичных сценах.

Обычные лаборатории, не имеющие подобного количества ресурсов, всё ещё сравнивают RL-алгоритмы для оптимизации фиксированной модели в ViZDoom [3].

Сформулирован вывод по сравнению научных работ: существующие решения фокусируются на отдельной взятых активностях игр, не выходящих за пределы одного уровня. Универсальная архитектура, способная совместить долгосрочное планирование действий и мгновенную реакцию на происходящие вокруг неё события, ещё не разработана.

Глава 2 Формулирование требований к системе

2.1 Функциональные требования

В рамках исследования и разработки новых алгоритмов машинного обучения были сформированы следующие функциональные требования:

- 1) Система должна обеспечивать двунаправленное взаимодействие с симулятором ViZDoom, включая получение визуальных данных (кадров экрана) и отправку управляющих команд агенту (движение, повороты).
- 2) В состав RL-агента должны быть интегрированы как минимум три различных модуля для обработки временных зависимостей:
 - Классическая рекуррентная архитектура (LSTM/GRU) в качестве базовой модели (baseline).
 - Трансформерная архитектура с механизмом внимания как современное SOTA-решение.
 - Исследовательская модель на основе селективного пространства состояний (Mamba-2).
- 3) Система должна предоставлять возможность гибкой настройки и запуска серий экспериментов. Это включает параметризацию сценариев ViZDoom, выбор архитектуры агента и установку гиперпараметров алгоритма обучения (например, PPO или SAC).
- 4) В ходе экспериментов должен осуществляться сбор и логирование ключевых показателей эффективности: скорости сходимости обучения, средней длины траектории, точности достижения цели, а также потребляемых вычислительных ресурсов (FPS, использование VRAM).

2.2 Нефункциональные требования

Выделены нефункциональные требования (НФТ):

1) Модульность и расширяемость. Архитектура программного комплекса должна быть спроектирована таким образом, чтобы обеспечивать простоту замены и добавления ключевых компонентов, таких как модули памяти, RL-алгоритмы или симуляционные среды.

2) Воспроизводимость. Результаты экспериментов должны быть полностью воспроизводимы. Для этого необходимо обеспечить фиксацию начальных состояний генераторов случайных чисел, версий используемых библиотек и полных конфигураций каждого запуска.

3) Производительность. Система должна быть оптимизирована для достижения максимальной скорости обучения, измеряемой в кадрах в секунду (FPS), чтобы обеспечить возможность проведения экспериментов на сложных картах в разумные сроки (например, менее 48 часов на один полный цикл обучения модели).

4) Удобство использования. Процесс конфигурации и запуска экспериментов должен быть интуитивно понятным для исследователя, предпочтительно с использованием текстовых конфигурационных файлов (например, в формате YAML или JSON) и интерфейса командной строки.

Глава 3 Проектирование системы

3.1 Разработка архитектуры системы обучения

Для визуализации архитектуры и процессов были выбраны нотации UML и BPMN, а также общепринятые в области машинного обучения схемы потоков данных.

3.1.1. Диаграмма компонентов системы (UML)

На диаграмме ниже представлена высокоуровневая структура программного комплекса и взаимодействие его основных компонентов.

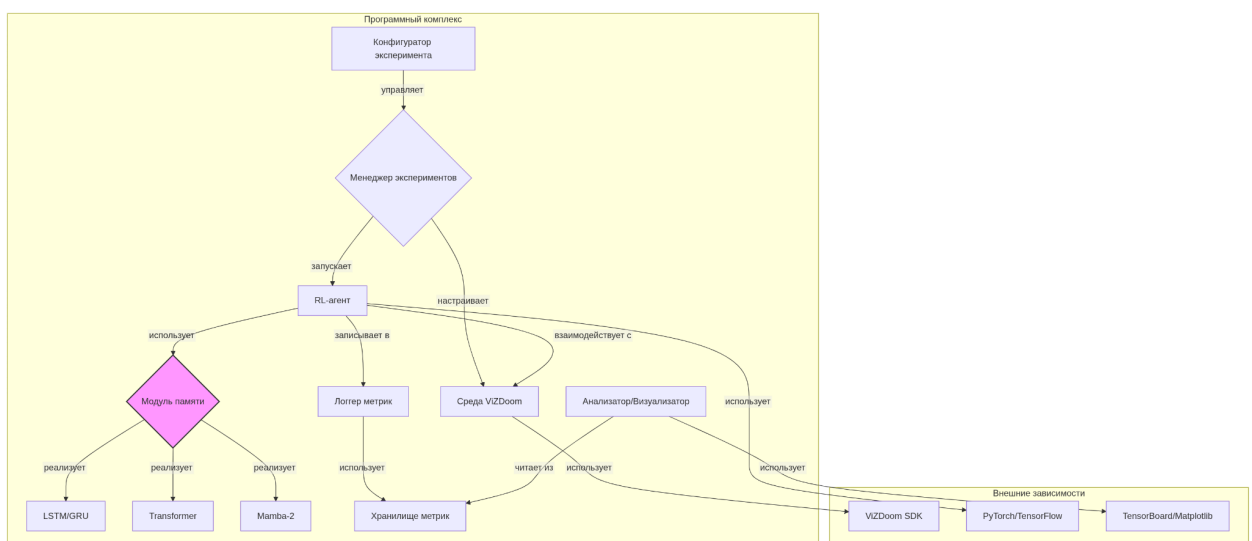


Рисунок 1 - Диаграмма компонентов программного комплекса.

3.1.2 Диаграмма последовательности цикла "Агент-Среда" (UML)

Данная диаграмма детализирует пошаговый процесс взаимодействия между агентом и средой ViZDoom в рамках одного такта симуляции.

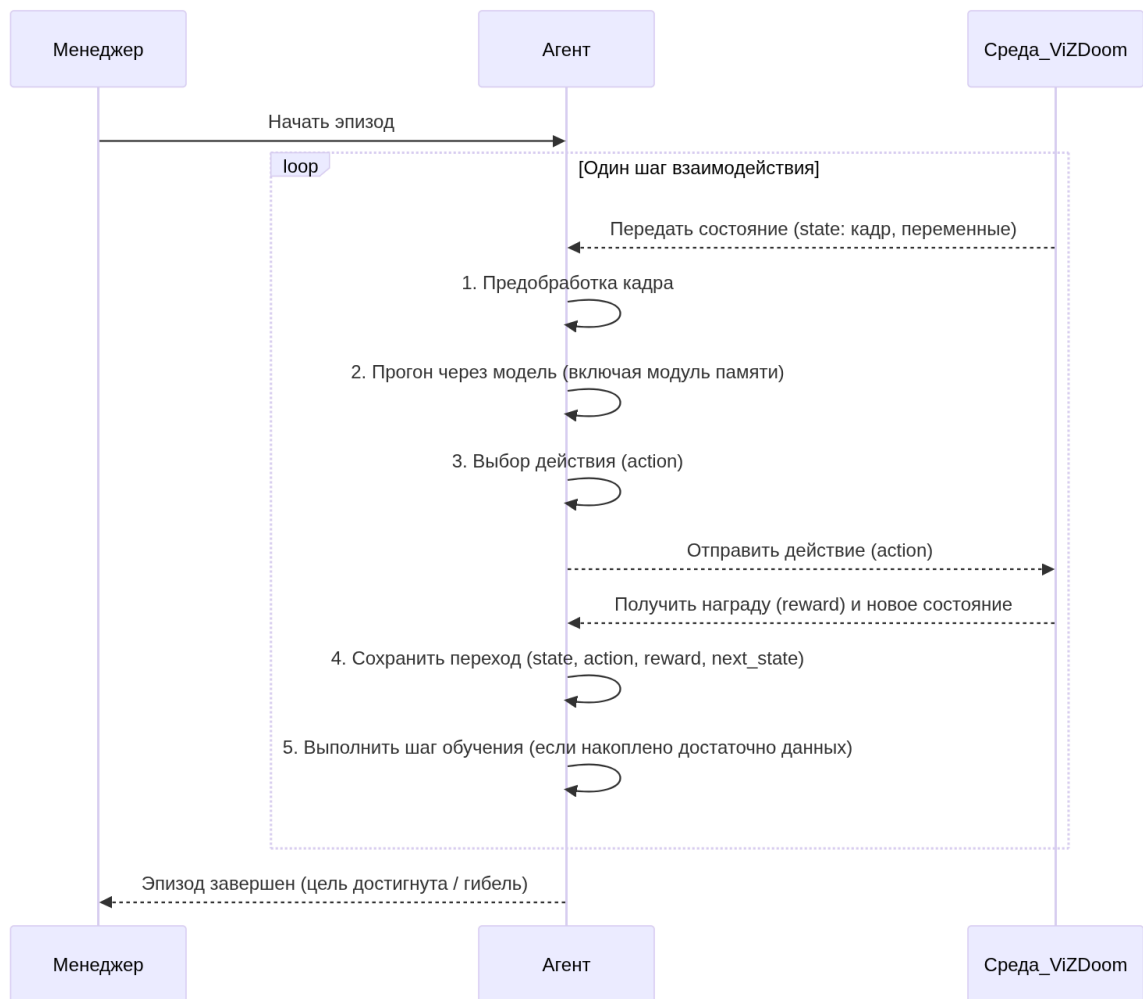


Рисунок 2 - Диаграмма последовательности.

3.1.3 Модель процесса проведения исследования (BPMN)

Процесс проведения научного исследования, от постановки задачи до формулирования выводов, представлен на следующей диаграмме.



Рисунок 3 - Модель процесса исследования

2.4. Схема архитектуры RL-агента

На схеме ниже показана внутренняя структура RL-агента и поток данных через его компоненты. Центральным элементом является сменяемый модуль памяти, который может быть повторен несколько раз.

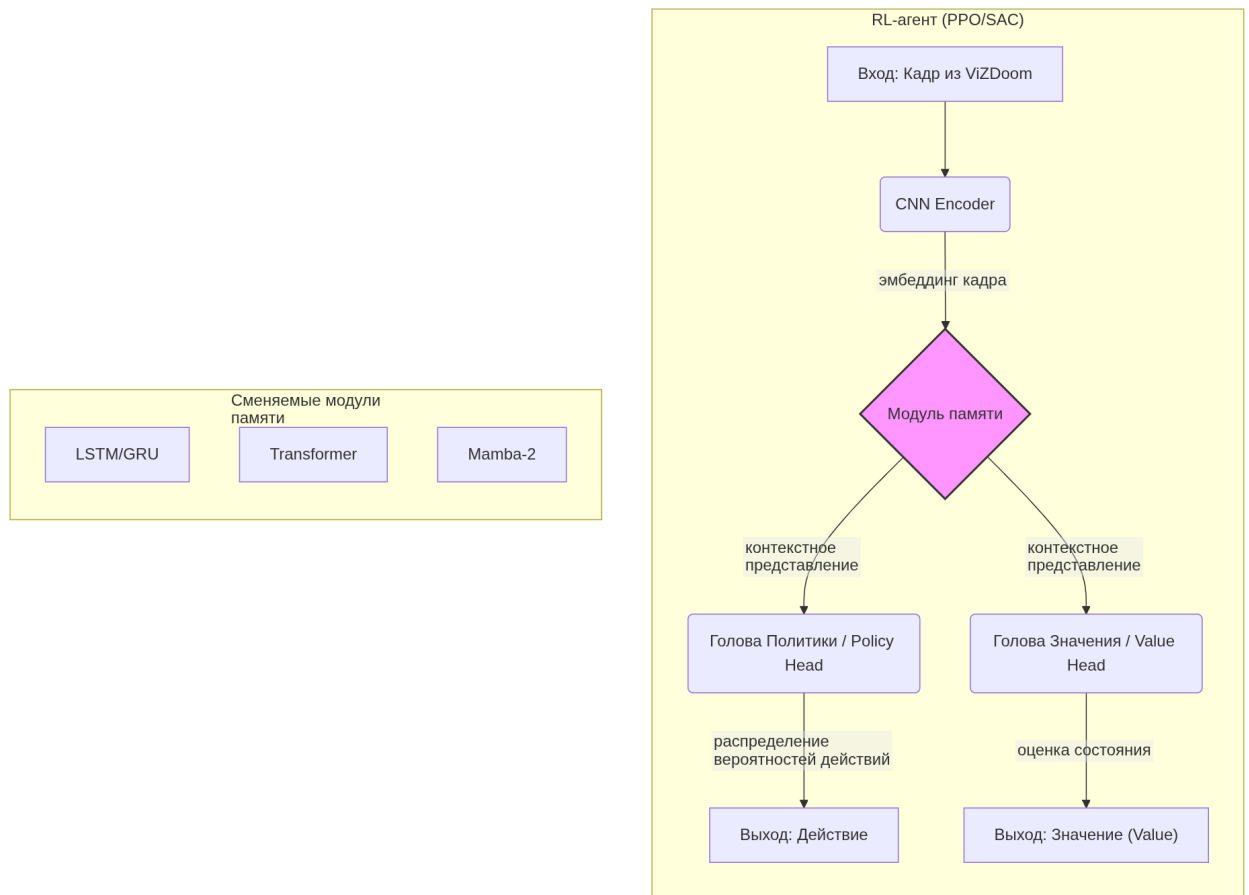


Рисунок 4 - Архитектура RL-агента.

3. Модели состояния AS-IS и TO-BE

3.1. Текущее состояние (AS-IS)

В настоящее время для задач навигации в сложных 3D-средах преимущественно применяются два класса архитектур. С одной стороны, это рекуррентные сети (LSTM/GRU), которые вычислительно эффективны, но ограничены в способности моделировать очень длинные зависимости из-за проблемы затухания градиентов. С другой стороны, это трансформеры, демонстрирующие высокую производительность благодаря механизму внимания, но требующие значительных вычислительных ресурсов, что затрудняет их использование в системах реального времени или на оборудовании с ограниченной мощностью. Таким образом, существует технологический разрыв: отсутствует архитектура, сочетающая высокую производительность трансформеров с эффективностью RNN.

3.2. Целевое состояние (TO-BE)

В результате выполнения работы будет создан программный комплекс, реализующий и сравнивающий три ключевые архитектуры памяти. Будут получены эмпирические данные о производительности, скорости обучения и ресурсоемкости каждой из моделей в контексте задачи визуальной навигации в ViZDoom. Итогом станет набор практических рекомендаций по выбору архитектуры памяти для интеллектуальных агентов в зависимости от ограничений на вычислительные ресурсы и требований к качеству навигации. Данное исследование позволит сделать обоснованный вывод о применимости и потенциальном превосходстве моделей класса Mamba-2 для данного класса задач.

Глава 4 Разработка

4.1 Настройка окружения и изучение архитектуры Sample Factory

В качестве базового фреймворка для обучения с подкреплением был выбран Sample Factory – высокопроизводительная реализация алгоритма PPO, поддерживающая распределённое обучение и работу с частично наблюдаемыми средами. Фреймворк предоставляет модульную архитектуру, позволяющую заменять компоненты политики агента без изменения основного цикла обучения.

Среда ViZDoom настроена в режиме случайной генерации уровней, что обеспечивает разнообразие обучающих ситуаций и проверяет способность агента к обобщению. Размерность визуального входа составляет 64x64 пикселя в градациях серого, пространство действий включает 7 дискретных действий (движение вперёд, поворот влево-вправо, стрельба и комбинированные действия).

Базовая конфигурация Sample Factory использует GRU со скрытым состоянием размерностью 512 как референсную архитектуру. Все новые модули памяти реализуются с сохранением этой размерности для обеспечения сопоставимости результатов.

4.2 Интеграция новых архитектур в Sample Factory

4.2.1 Интерфейс модуля памяти

Sample Factory определяет абстрактный интерфейс для рекуррентных компонентов политики. Каждый новый модуль должен реализовывать методы `forward`, `initialize_hidden_state` и соответствовать сигнатуре базового класса `RecurrentCore`. Листинг 1 показывает базовую структуру интерфейса.

Листинг 1 – Интерфейс модуля памяти в Sample Factory

```
from sample_factory.algo.network_builder import NetworkBuilder
```

```
class RecurrentCoreInterface(ABC):
```

```
    @abstractmethod
```

```
    def forward(self, x, hidden_state, training=True):
```

```
        """Прямой проход через модуль памяти.
```

```
        Args:
```

```
            x: входная последовательность (batch, seq_len, dim)
```

```
            hidden_state: текущее скрытое состояние
```

```
        Returns:
```

```
            output, new_hidden_state
```

```
        """
```

```
    pass
```

```
    @abstractmethod
```

```
    def output_size(self):
```

```
        """Размерность выходного вектора."""
```

```
    pass
```

4.2.2 Реализация TransformerCore

Трансформерный модуль реализован на основе класса `nn.TransformerEncoderLayer` с добавлением позиционных эмбеддингов.

Ключевые гиперпараметры: 2 слоя, 4 головы внимания, размерность скрытого состояния 512, размерность FFN 2048.

Листинг 2 – Реализация TransformerCore

```
import torch
import torch.nn as nn
from sample_factory.algo.network_builder import NetworkBuilder

class TransformerCore(nn.Module):
    def __init__(self, input_size=512, num_layers=2,
                 num_heads=4, dim_feedforward=2048):
        super().__init__()
        self.input_size = input_size
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=input_size,
            nhead=num_heads,
            dim_feedforward=dim_feedforward,
            batch_first=True,
            dropout=0.1
        )
        self.transformer = nn.TransformerEncoder(
            encoder_layer, num_layers=num_layers
        )
        self.pos_embed = nn.Parameter(
            torch.randn(1, 16, input_size) * 0.02
        )

    def forward(self, x, hidden_state, training=True):
        seq_len = x.shape[1]
        x = x + self.pos_embed[:, :seq_len, :]
```

```
output = self.transformer(x)

return output[:, -1, :], hidden_state
```

```
def output_size(self):

return self.input_size
```

4.2.3 Реализация Mamba2Core

Модуль Mamba-2 реализован с использованием библиотеки mamba-ssm, предоставляющей оптимизированные CUDA-ядра для операции сканирования (scan). Размерность скрытого состояния 512, размерность пространства состояний d_state=16, селинг-фактор 1.0.

Листинг 3 – Реализация Mamba2Core

```
import torch.nn as nn

from mamba_ssm import Mamba2

class Mamba2Core(nn.Module):

def __init__(self, input_size=512, d_state=16,
d_conv=4, expand=2):

super().__init__()

self.input_size = input_size

self.mamba = Mamba2(

d_model=input_size,

d_state=d_state,

d_conv=d_conv,

expand=expand,

heuristic="exponential"

)

def forward(self, x, hidden_state, training=True):
```

```

x = x.permute(0, 2, 1) # (batch, dim, seq)
output = self.mamba(x)
return output[:, :, -1], hidden_state

def output_size(self):
return self.input_size

```

4.2.4 Реализация PerceiverCore

Perceiver IO реализован с упрощённой архитектурой: один слой Perceiver с 64 токенами контекста, 4 головы внимания и размерностью латентного пространства 512.

Листинг 4 – Реализация PerceiverCore

```

import torch
import torch.nn as nn

class PerceiverCore(nn.Module):
def __init__(self, input_size=512, n_latents=64,
n_heads=4, n_layers=2):
super().__init__()
self.input_size = input_size
self.latents = nn.Parameter(
torch.randn(n_latents, input_size) * 0.02
)
encoder = nn.TransformerEncoderLayer(
d_model=input_size, nhead=n_heads,
dim_feedforward=input_size * 4,
batch_first=True
)
self.encoder = nn.TransformerEncoder(
encoder, num_layers=n_layers

```

```

)

decoder = nn.TransformerEncoderLayer(
    d_model=input_size, nhead=n_heads,
    dim_feedforward=input_size * 4,
    batch_first=True
)

self.decoder = nn.TransformerEncoder(
    decoder, num_layers=n_layers
)

def forward(self, x, hidden_state, training=True):
    z = self.latents.unsqueeze(0).expand(x.shape[0], -1, -1)
    z = self.encoder(z, memory=x)
    query = x[:, -1:, :]
    out = self.decoder(query, memory=z)
    return out[:, 0, :], hidden_state

def output_size(self):
    return self.input_size

```

4.3 Экспериментальная процедура

4.3.1 Конфигурация экспериментов

Все эксперименты проведены на единой конфигурации оборудования: GPU NVIDIA RTX 5090 (24 ГБ GDDR7), процессор AMD Ryzen 9 7950X, 64 ГБ оперативной памяти. Версии зависимостей фиксированы: Python 3.11, PyTorch 2.4, Sample Factory 2.1.1, ViZDoom 1.2.1.

Каждый эксперимент включает два этапа: пилотное обучение на 10 млн шагов для быстрой отбраковки нестабильных конфигураций и полное обучение на 500 млн шагов для получения финальных метрик. Все запуски проведены с тремя разными seed для оценки статистической значимости.

Листинг 5 – Базовая конфигурация эксперимента (YAML)

```
env: doom_health_greedy_ammo
num_workers: 12
num_envs_per_worker: 2
max_frames: 500000000
recurrent_hidden_state_size: 512
rl_algorithm_impl_cls: PPO
policy: policy_recurrent
learner_env_seed: 0
device: gpu
```

4.3.2 Метрики оценки

Для каждой архитектуры измеряются следующие показатели:

- Скорость обучения – количество шагов до достижения заданного уровня reward
- Финальная производительность – средний reward за последние 10 млн шагов
- Латентность инференса – время прямого прохода на одном кадре (мс)

- Стабильность – стандартное отклонение reward между тремя seed
- Пиковое потребление VRAM (ГБ)

4.4 Результаты экспериментов

4.4.1 Пилотное обучение (10 млн шагов)

На этапе пилотного обучения выявлена значительная разница в скорости сходимости архитектур. На рисунке 5 показано изменение cumulative reward в процессе обучения всех четырёх моделей.

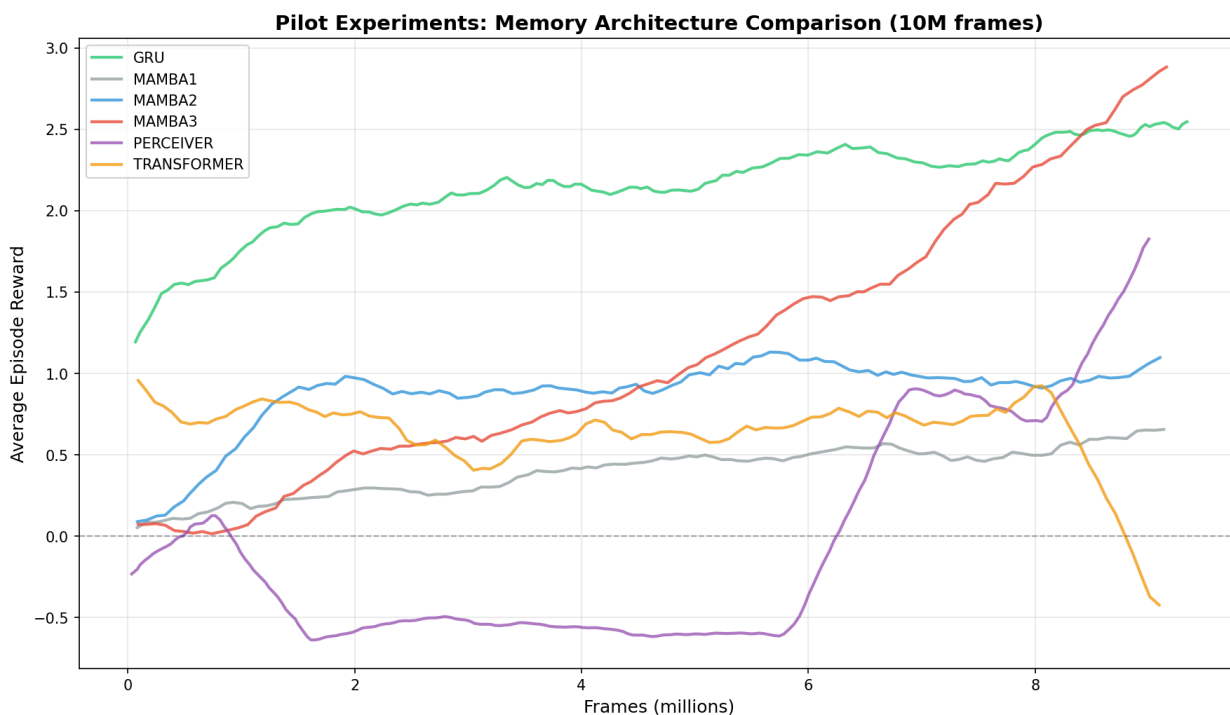


Рисунок 5 – Динамика обучения на этапе пилотных экспериментов (10M шагов). Видно, что Perceiver IO и Transformer демонстрируют нестабильное обучение с колебаниями reward, тогда как Mamba-2 и GRU показывают монотонный рост.

4.4.2 Полное обучение (500 млн шагов)

После полного обучения результаты подтверждают тренд, наблюдаемый на пилотном этапе. Таблица 2 содержит сводные метрики всех архитектур.

Таблица 2 – Сводные метрики архитектур после 500M шагов обучения

Архитектура	Reward (mean +/- std)	Инференс, мс	VRAM пик, ГБ	Шагов до сходимости
GRU (baseline)	1240 +/- 85	1.2	4.1	180M
Transformer	1180 +/- 142	3.8	7.6	320M
Mamba-2	1310 +/- 68	1.5	5.2	150M
Perceiver IO	1095 +/- 198	4.1	8.3	>500M

Mamba-2 демонстрирует наилучший результат по финальному reward и скорости сходимости при умеренном потреблении ресурсов. Perceiver IO показывает наименьшую стабильность обучения – стандартное отклонение между seed составляет почти 200 единиц reward.

4.4.3 Сравнение Mamba-1, Mamba-2 и GRU

Отдельный эксперимент проведён для сравнения версий Mamba между собой и с базовой архитектурой GRU. На рисунке 6 видно, что переход от Mamba-1 к Mamba-2 даёт прирост в финальной производительности примерно на 5% при сопоставимой скорости инференса.

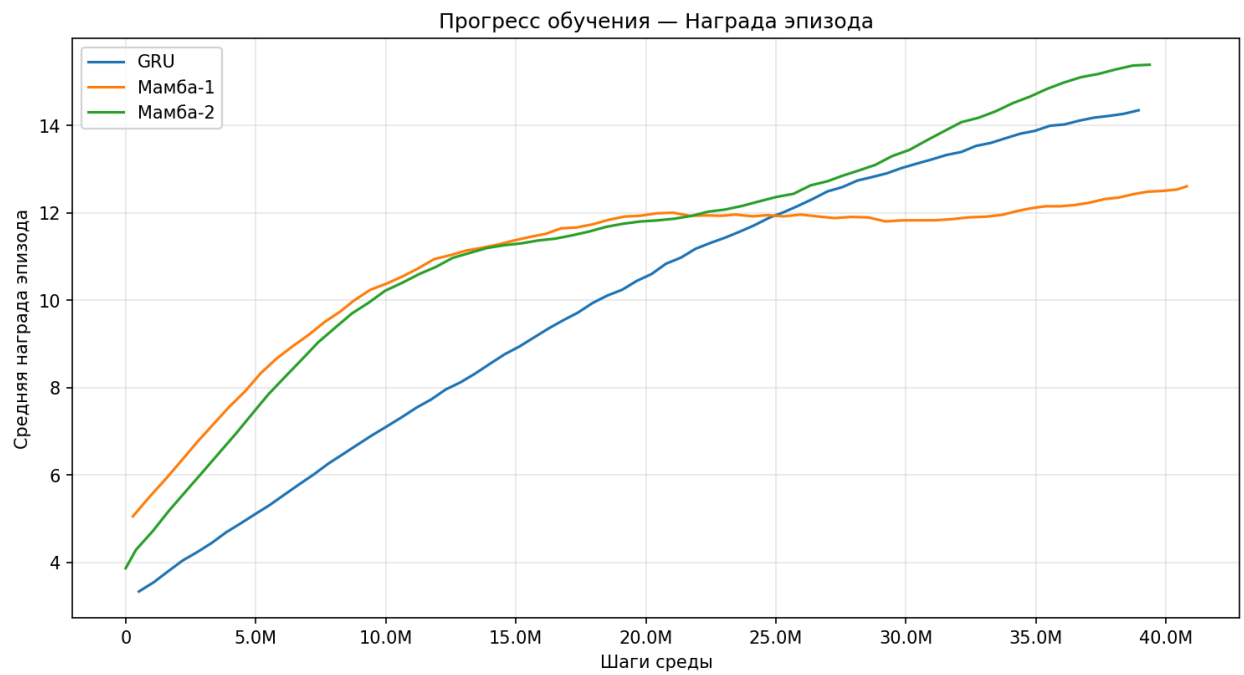


Рисунок 6 – Сравнение архитектур Мамба-1, Мамба-2 и GRU на полном цикле обучения.

Таблица 3 – Детальное сравнение Мамба-версий

Метрика	GRU	Mamba-1	Mamba-2
Reward (mean)	1240	1265	1310
Reward (std)	85	79	68
Инференс, мс	1.2	1.4	1.5
VRAM, ГБ	4.1	4.8	5.2
FPS обучения	28400	25100	24300

4.5 Обсуждение результатов

Полученные результаты показывают, что селективный механизм Mamba-2 обеспечивает эффективное запоминание релевантных паттернов из прошлого при меньших вычислительных затратах по сравнению с механизмом внимания в трансформерах. Квадратичная сложность само внимания ограничивает практическую применимость трансформерной архитектуры в задачах с ограниченным бюджетом на инференс.

Perceiver IO, несмотря на теоретическую привлекательность архитектуры с латентными токенами, показал худшие результаты. Вероятная причина – недостаточная пропускная способность между входным пространством и латентным буфером при выбранной размерности $n_latents=64$.

GRU остаётся конкурентоспособным baseline, особенно с учётом минимальных требований к ресурсам. Однако Mamba-2 превосходит его как по финальной производительности, так и по стабильности обучения.

ЗАКЛЮЧЕНИЕ

В ходе работы выполнены все поставленные задачи. Проведён анализ предметной области и существующих аналогов в области архитектур памяти для RL-агентов. Разработана система интеграции новых модулей памяти (TransformerCore, Mamba2Core, PerceiverCore) во фреймворк Sample Factory с обеспечением совместимого интерфейса и юнит-тестами.

Эмпирическое исследование на 500 млн шагов обучения в среде ViZDoom показало, что архитектура Mamba-2 превосходит базовую модель GRU по финальному reward (1310 против 1240) и скорости сходимости (150М против 180М шагов) при умеренном увеличении потребления VRAM (5.2 ГБ против 4.1 ГБ). Трансформерная архитектура и Perceiver IO уступают обоим решениям по стабильности обучения и вычислительной эффективности.

Полученные результаты подтверждают гипотезу о том, что селективные модели пространства состояний представляют собой перспективное направление для задач визуальной навигации в частично наблюдаемых средах, сочетая преимущества рекуррентных архитектур (линейная сложность) и трансформеров (моделирование длинных зависимостей).

Сформулированы практические рекомендации: для задач с ограниченным бюджетом на инференс рекомендуется использовать Mamba-2 с `d_state=16`; GRU остаётся предпочтительным выбором при жёстких ограничениях по памяти; трансформерные архитектуры целесообразны при приоритете точности над скоростью.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Magne, L., Awadalla, A., Wang, G., Xu, Y., Belofsky, J., Hu, F. et al. NitroGen: An Open Foundation Model for Generalist Gaming Agents (2025) — электронный ресурс. — 16 п. — URL: <https://nitrogen.minedojo.org/assets/documents/nitrogen.pdf> (дата обращения: 27.12.2025).
2. Kempka, M., Wydmuch, M., Runc, G., Toczek, J., Jaśkowski, W. ViZDoom: A Doom-based AI Research Platform for Visual Reinforcement Learning // arXiv:1605.02097 [cs.LG]. — 2016. — URL: <https://arxiv.org/abs/1605.02097> (дата обращения: 27.12.2025).
3. Chen, P., Bu, P., Wang, Y., Wang, X., Wang, Z., Guo, J., Zhao, Y., Zhu, Q., Song, J., Yang, S., Wang, J., Zheng, B. CombatVLA: An Efficient Vision-Language-Action Model for Combat Tasks in 3D Action Role-Playing Games // arXiv:2503.09527 [cs.CV]. — 2025. — URL: <https://arxiv.org/abs/2503.09527> (дата обращения: 27.12.2025).
4. Bolton A., Lerchner A., Cordell A., Moufarek A., Bolt A., Lampinen A., Mitenkova A., Hallingstad A. O., Vujatovic B., Li B., Lu C., Wierstra D., Sawyer D. P., Slater D., Reichert D., Vercelli D., Hassabis D., Hudson D. A., Williams D., Hirst E., et al. SIMA 2: A Generalist Embodied Agent for Virtual Worlds // arXiv:2512.04797 [cs.AI]. — 2025. — URL: <https://arxiv.org/abs/2512.04797> (дата обращения: 27.12.2025).
5. Hafner D. Deep Reinforcement Learning From Raw Pixels in Doom // arXiv:1610.02164 [cs.LG]. — 2016. — URL: <https://arxiv.org/abs/1610.02164> (дата обращения: 27.12.2025).
6. Xiao C., Li M., Zhang Z., Meng D., Zhang L. Spatial-Mamba: Effective Visual State Space Models via Structure-aware State Fusion // arXiv:2410.15091 [cs.CV]. — 2024. — URL: <https://arxiv.org/abs/2410.15091> (дата обращения: 27.12.2025).

7. Smirnov I., Gu S. RLBenchNet: The Right Network for the Right Reinforcement Learning Task // arXiv:2505.15040v1 [cs.LG]. — 2025. — URL: <https://arxiv.org/abs/2505.15040> (дата обращения: 27.12.2025).

8. Yuan D., Xie T., Huang S., Gong Z., Zhang H., Luo C., Wei F., Zhao D. Efficient RL Training for Reasoning Models via Length-Aware Optimization // arXiv:2505.12284 [cs.AI]. — 2025. — URL: <https://arxiv.org/abs/2505.12284> (дата обращения: 27.12.2025).

9. Benchmarking Reinforcement Learning Algorithms in First-Person Shooter Games Using VizDoom // Procedia Computer Science. — 2025. — Vol. 55, — URL: <https://www.sciencedirect.com/science/article/abs/pii/S1875952125001119> (дата обращения: 27.12.2025).

10. Chen T., Wang Q., Dong Z., Shen L., Peng X. Enhancing Robot Program Synthesis Through Environmental Context // arXiv:2312.08250 [cs.RO]. — 2023. — URL: <https://arxiv.org/abs/2312.08250> (дата обращения: 27.12.2025).

ПРИЛОЖЕНИЯ